

Credit card recommendation use-case

Why not the credit card recommendation not be multi-label?

at first glance, multi-label classification might seem like a fit, but it's not appropriate for your credit card recommendation use case. Let's break down why in a structured way:

✓ First, What is Multi-label Classification?

Multi-label classification is used when multiple labels (classes) can be true for the same instance.

Example:

- An image can be labeled both ["cat", "outdoor", "night"]
- A news article might belong to ["finance", "economy", "AI"]

✓ It's "multi-label" because more than one label can be simultaneously valid and correct.

⊘ Why Not Multi-label for Credit Card Recommendation?

● 1. Users Can Apply for Only ONE Card at a Time

- Your problem is about recommending the single best credit card (or top-N ranked).
- Users don't apply for multiple cards at once, especially not from the same bank.
- So, the label is one card per customer, not multiple.

➡ Multiclass or ranking is a better match.

● 2. Multi-label means independent outputs

- In multi-label, each output is treated independently:
 - "Is CC1 a good fit?" ✓
 - "Is CC2 a good fit?" ✓
- But in recommendation, you want to compare and rank all valid cards — not treat each as an independent yes/no.

➡ Ranking gives relative preference, multi-label does not.

- 3. Business constraint: recommendation, not assignment
 - You're recommending the best eligible card, not assigning cards.
 - You don't want to say "the user qualifies for all of CC3, CC5, CC7" — even if they do — you want to pick the best one (max revenue, acceptance, fit).

➔ This is a ranking or multiclass setup, not multi-label.

Quick Comparison

Feature	Multiclass	Multi-label	Ranking
Predict 1 best card?	✅ Yes	❌ No (predicts 0/1 per class)	✅ Yes
Predict multiple cards?	❌ No	✅ Yes	✅ Can return Top-N
Use relative preference?	❌ No	❌ No	✅ Yes
Suitable for your use case?	✅ Yes (if 1 label per user)	❌ No	✅✅ Best fit

↓

how the training data would look like for customer and credit card to perform multiclass classification ?

To perform multiclass classification for your credit card recommendation system, your training dataset needs to be structured such that:

Each row = one customer record

Target (label) = one best credit card class (e.g., CC1 to CC10)

Customer_ID	Age	Income	Credit_Score	Spending	Current_Card	Tenure	Target_Card (Label)
CUST1	35	50000	750	20000	CC10	2 yrs	CC8
CUST2	45	120000	820	90000	CC5	4 yrs	CC3
CUST3	26	30000	690	10000	CC10	1 yr	CC9
CUST4	60	180000	860	150000	CC4	5 yrs	CC1

✅ Columns Explained

Column	Description
Customer_ID	(Optional) Identifier – drop before modeling
Age	Customer demographic
Income	Annual income
Credit_Score	FICO or internal score
Spending	Yearly or monthly spending
Current_Card	Categorical (e.g., CC10)
Tenure	Time with bank / current card
Target_Card	🟡 Multiclass label: ONE upgraded card

$f(\text{age, income, score, spending, current_card, ...}) \rightarrow \text{Target_Card (one of CC1 to CC10)}$

Step	How
Encode Current_Card	Use LabelEncoder or OneHotEncoding
Scale Income, Spending	Use MinMaxScaler or StandardScaler
Encode Target_Card	LabelEncode (e.g., CC1=0, ..., CC10=9)

In real world, the customer data is differently stored and credit card data is differently stored? How do we combine the label in training data, because the customer only has current card but target should be calculated from his eligibility, spending habits and credit score, how to do this?

In the real world:

- ✅ Customer data and credit card product data are stored in different systems/tables
- ❌ There's no "target card" column readily available
- ✅ You need to generate the training labels intelligently, either:

- From historical upgrades
- Or by applying business logic + eligibility + profit models

How to create training data with:

Features: customer profile + current card

Label: the card they should upgrade to (target card)

✅ **Business Rule–Driven Labels**

Generate labels based on spending + eligibility + revenue

Steps:

1. Join customer profile + card catalog
2. Apply eligibility filter:
 - $\text{income} \geq \text{min_income}$
 - $\text{credit_score} \geq \text{threshold}$
 - $\text{current_card_tier} < \text{candidate_card_tier}$
3. From eligible cards:
 - Select card with highest expected revenue, or
 - Just 1 or 2 levels higher

How to generate labels using Business Rule–Driven Labels?

To generate labels using **Option B: Business Rule–Driven Labels** for your credit card recommendation system, you simulate what a “**smart business analyst**” or a **rule engine** would decide based on:

Eligibility rules (who qualifies for which card)

Upgrade logic (which card is higher-tier than the current one)

Business goals (maximize expected revenue, margin, etc.)

💡 **Goal**

For each customer, choose a **target credit card** from the card catalog using:

- Eligibility criteria (e.g. income, credit score)
- Business logic (e.g. pick top 1 eligible card based on revenue)

Step 1: Sample Data Setup

- ♦ Customer Data (customers.csv)

customer_id	age	income	credit_score	current_card_id
C1	30	60000	740	CC10
C2	45	120000	810	CC7
C3	28	35000	680	CC10

- ♦ Card Catalog (cards.csv)

card_id	tier	min_income	min_credit_score	annual_fee	expected_revenue
CC1	1	150000	800	500	5000
CC2	2	120000	780	400	4500
CC3	3	100000	750	300	4000
CC4	4	90000	720	250	3500
CC5	5	80000	700	200	3000
CC6	6	60000	680	150	2500
CC7	7	40000	650	100	2000
CC8	8	30000	640	50	1500
CC9	9	20000	600	25	1000
CC10	10	10000	550	0	500

Step 2: Load and Join Data

```
import pandas as pd

customers = pd.read_csv("customers.csv")
cards = pd.read_csv("cards.csv")

# Add current card tier to customer table
customers = customers.merge(cards[['card_id', 'tier']],
                           left_on='current_card_id',
                           right_on='card_id',
                           how='left') \
    .rename(columns={'tier': 'current_card_tier'}) \
    .drop(columns=['card_id'])
```

Step 3: Label Generation Function

```
def generate_target_card(customer_row, cards_df):
    eligible = cards_df[
        (cards_df['min_income'] <= customer_row['income']) &
        (cards_df['min_credit_score'] <= customer_row['credit_score']) &
        (cards_df['tier'] < customer_row['current_card_tier']) # only upgrades
    ]

    if eligible.empty:
        return customer_row['current_card_id'] # fallback: no upgrade

    # Pick card with highest expected revenue (can change Logic here)
    best_card = eligible.sort_values('expected_revenue', ascending=False).iloc[0]
    return best_card['card_id']
```

Step 4: Apply Rule to All Customers

```
customers['target_card'] = customers.apply(
    lambda row: generate_target_card(row, cards), axis=1
)
```

Final Training Data

customer_id	income	credit_score	current_card_id	current_card_tier	target_card
C1	60000	740	CC10	10	CC6
C2	120000	810	CC7	7	CC5
C3	35000	680	CC10	10	CC8

You now have training data in multiclass format:

X = customer features

y = target_card